PropMatch

NoSQL / Vector Component Technical Report

Project: Real-Estate Property Listing and Recommendation Application
NoSQL database: MongoDB **Database name:** propmatch_nosql

1. Introduction and Scope

PropMatch combines a relational database with MongoDB. The MongoDB component is used for flexible buyer-preference records, user-activity events, and cached property recommendations. This report documents the data model, collections and documents, indexes, embeddings/vector-search status, metadata, and retrieval strategy using the supplied **mongo-setup.js** and **mongo-seed.js** scripts, plus the recommendation API behavior reviewed for this project.

2. NoSQL Data Model

MongoDB follows a document-oriented model. Data is stored as BSON documents inside collections rather than as rows connected through relational foreign keys. PropMatch uses the following logical model:

```
propmatch_nosql\n|-- user_preferences       (buyer search preferences)\n|-- user_activity          (buyer interaction event
```

The collections are associated through application identifiers. **userId** identifies the buyer across preference, activity, and recommendation records. **propertyId** associates activity and recommendation items with property records managed by the application. The MongoDB scripts do not define relational foreign-key constraints; the application is responsible for using these identifiers consistently.

3. Collections and Document Structures

3.1 user_preferences

Purpose: stores a buyer's property-search preferences. The setup script defines a JSON Schema validator requiring userId, listingType, budgetMin, budgetMax, preferredCities, preferredLocalities, preferredTypeIds, minBedrooms, furnishing, mustHaveAmenityIds, createdAt, and updatedAt. It constrains userId and preference arrays to the specified BSON types, and permits null values for selected optional preference values.

```
{
  "userId": NumberInt(5),
  "listingType": "RENT",
  "budgetMin": 30000,
  "budgetMax": 60000,
  "preferredCities": ["Bengaluru"],
  "preferredLocalities": [],
  "preferredTypeIds": [],
  "minBedrooms": 2,
  "furnishing": null,
  "mustHaveAmenityIds": [],
  "createdAt": ISODate("..."),
  "updatedAt": ISODate("...")
}
```

The JSON above illustrates the validator's intended flat document structure. The supplied seed script currently inserts a different shape: a nested `preferences` object containing city, listingType, minPrice, maxPrice, minBedrooms, preferredPropertyTypes, and preferredAmenities, plus updatedAt. It omits several required top-level fields. This discrepancy is described in Section 9.

3.2 user_activity

Purpose: records buyer interactions with the application. The validator requires userId, eventType, and createdAt. propertyId may be an integer, long, or null. eventType is restricted to VIEW, SEARCH, FAVORITE, UNFAVORITE, INQUIRY, or VISIT_REQUEST. An optional metadata object stores event-specific context.

```
{
  "userId": NumberInt(5),
  "propertyId": NumberInt(6),
  "eventType": "VIEW",
  "metadata": { "source": "recommendations" },
  "createdAt": ISODate("...")
```

```
}
```

The seed script inserts five activity events: property views, a favorite action, and a search event. Metadata examples include the source of a view and search filters such as city, listing type, and price range.

3.3 recommendations_cache

Purpose: stores generated recommendation results for a buyer. The setup validator requires userId, items, and generatedAt. Each element of items must be an object containing propertyId, score, and reasons, where reasons is an array of strings.

```
{
  "userId": NumberInt(5),
  "items": [
    {
      "propertyId": NumberInt(6),
      "score": 0.95,
      "reasons": [
        "Matches preferred city, rent range and bedrooms"
      ]
    }
  ],
  "generatedAt": ISODate("...")
}
```

This is the validator's intended structure. The supplied seed script instead writes a `recommendations` array containing a singular string field `reason`. That does not match the validator's required `items` and `reasons` fields.

4. Indexes and Their Purpose

The setup script explicitly creates the following indexes:

Collection	Index definition	Purpose
user_preferences	{ userId: 1 }, unique	Fast buyer lookup; prevents more than one preference document per user.
user_activity	{ userId: 1, createdAt: -1 }	Retrieves a buyer's activity, newest first.
user_activity	{ propertyId: 1, eventType: 1 }	Supports filtering events by property and event type.
user_activity	{ createdAt: 1 }, TTL 15552000	Expires activity records after 15,552,000 seconds (180 days).
recommendations_cache	{ userId: 1 }, unique	Fast cache lookup; enforces one cache document per user.
recommendations_cache	{ generatedAt: 1 }, TTL 3600	Expires cache documents approximately one hour after generatedAt.

TTL indexes are MongoDB's automatic document-expiration mechanism. The activity TTL is 180 days; the recommendation-cache TTL is 3,600 seconds (one hour). TTL deletion is performed asynchronously by MongoDB, so application code should still check whether a cache entry is fresh before using it.

5. Embeddings and Vector Search

The supplied MongoDB setup and seed scripts do **not** define embedding fields, vector embeddings, vector indexes, a vector search index, or an ANN (approximate-nearest-neighbor) retrieval operation. The reviewed recommendation flow is based on buyer preferences/activity and a cached recommendation result. Therefore, vector embeddings and semantic/vector retrieval are **not implemented or evidenced in the supplied project files**.

For an accurate academic submission, PropMatch should be described as a MongoDB-backed preference/activity recommendation system, not as a vector-search system. If vector functionality is added later, the report should document the embedding model, vector dimensions, source text/features embedded, storage field, vector index configuration, similarity metric, and query/retrieval implementation. Those details cannot be reported as current features.

6. Metadata

Metadata gives context to stored records and supports filtering, interpretation, and traceability. The project scripts demonstrate these metadata elements:

```
userId       -> buyer associated with the record\npropertyId   -> property associated with an event or recommendation\neven
```

The seed script uses NumberInt for user/property identifiers and Date values for timestamps. Activity metadata includes examples such as source="recommendations", source="search", and nested search filters. The recommendation validator also anticipates a numeric score and an array of explanation strings for each result.

7. Retrieval Strategy

The recommendation retrieval flow reviewed for PropMatch is a cache-first application strategy:

```
1. Authenticate the request and confirm the user's role is BUYER.\n2. Identify the buyer using the authenticated userId.\n3
```

This combines application-level freshness checking with the one-hour MongoDB TTL index. The TTL index supports cleanup; the API's timestamp condition determines whether a result is fresh enough to return. The user_activity and user_preferences collections provide the flexible data model for buyer preferences and interaction history. The exact internal scoring formula should be described from the recommender source code, not inferred from the MongoDB scripts.

8. Data Validation and Integrity

The setup script attaches JSON Schema validators to all three collections with validationLevel="moderate" and validationAction="warn". Under this configuration, documents that fail validation produce warnings rather than being rejected. This makes it especially important to align the seed data with the validator definitions.

9. Implementation Issues Identified in the Supplied Scripts

Issue 1 — user_preferences: the validator expects a flat document with multiple required top-level fields and createdAt/updatedAt. The seed documents instead use a nested preferences object and omit several required fields.

Issue 2 — recommendations_cache: the validator expects an items array with reasons as an array of strings. The seed document instead uses recommendations and a singular reason string.

Because validationAction is set to warn, these mismatches may still allow the seed writes while generating validation warnings. Update either the seed documents or validators so that the chosen schema is consistent. The report intentionally does not claim that the seed examples conform to the validators.

10. MySQL and MongoDB Roles in PropMatch

MySQL serves as the relational store for structured application records. MongoDB stores flexible buyer preference documents, interaction events, and recommendation-cache documents. The application layer links related information through userId and propertyId. This hybrid design uses each database for a different data shape and access pattern.

11. Conclusion

PropMatch's NoSQL component consists of three MongoDB collections: user_preferences, user_activity, and recommendations_cache. The setup script defines JSON Schema validation, unique and compound indexes, and TTL indexes for activity and recommendation-cache retention. The recommendation API uses a cache-first retrieval strategy with a one-hour freshness check. Metadata includes user/property identifiers, event types, timestamps, scores, and recommendation explanations. Vector embeddings and vector search are not present in the supplied implementation and should be marked as not applicable/not implemented unless they are added and demonstrated.

Source basis: db/mongo-setup.js and db/mongo-seed.js supplied for this report, plus the previously reviewed recommendation API behavior. Example JSON documents above illustrate the validator's intended structure where explicitly noted; they are not represented as exact seed documents.